

DailyStack

Habit & Life Tracker — Product Design Specification

Developer Handoff Document · v1.0 · June 2026

Author: Matoke Brian Omwamba · github.com/mattobryan

This document is the complete product specification for **DailyStack**, a single-page habit and life tracking application. It covers system architecture, UI design tokens, component contracts, data models, business logic rules, and theme specifications. Intended audience: a front-end or full-stack developer taking this from prototype to production.

Product Overview

DailyStack is a single-page web application for daily habit tracking, goal setting, and personal metric logging. The system is built as a React SPA with `localStorage` persistence (upgradeable to a backend). It is designed to be used on desktop and mobile browsers without installation.

Design Philosophy

The visual language is inspired by high-density data dashboards: dark navy backgrounds, vibrant week-colour bands, inline checkbox grids, and analysis panels that sit on the same page as the tracker. A light mode variant uses a lavender-tinted off-white surface with identical information architecture.

The application is intentionally single-page: all modules (monthly grid, analysis, daily trackers, goals) are accessible via top-level tabs with zero page navigation.

Core Principles

- 1. Zero friction check-in.** Every habit is one tap/click. No modal, no confirmation.
- 2. Context-aware habits.** Friday-only habits (*Sabbath Prep*, *Weekly Review*) are displayed but non-interactive on other days, maintaining visual consistency.
- 3. Persistent state.** All data writes to `localStorage` on every interaction. No save button.
- 4. Theme parity.** Both dark and light modes expose identical information at identical density. No features are hidden by theme.
- 5. Goal-habit linking.** Goals may be standalone or linked to one or more habits, with duration-based progress tracking.

Information Architecture

Navigation Model

DailyStack has a single horizontal tab bar. Each tab renders a full-width panel in place. There is no routing, no back/forward navigation, no modal layers (except the inline goal form).

Tab	Key	Access	Primary Content
Monthly Grid	<code>grid</code>	Always	Week-banded checkbox grid + summary rows
Analysis	<code>analysis</code>	Always	Habit completion bars, weekly breakdown, 30d trend, mood chart
Daily Trackers	<code>trackers</code>	Always	Mood, water, sleep, Pomodoro timer, gratitude journal
Goals	<code>goals</code>	Always	Active goals list + new goal form

Habits Data Model

Habit Definition Schema

Each habit is a static configuration object. The full set is defined at `HABITS` constant in source.

Field	Type	Description
<code>id</code>	<code>string</code>	Unique snake_case identifier
<code>label</code>	<code>string</code>	Display name
<code>emoji</code>	<code>string</code>	Single emoji prefix
<code>category</code>	<code>enum</code>	<code>body</code> <code>work</code> <code>spirit</code> <code>mind</code>
<code>everyDay</code>	<code>boolean</code>	If <code>false</code> , habit is day-restricted
<code>dayOfWeek</code>	<code>0-6</code>	ISO weekday (0=Mon). Required if <code>everyDay=false</code>

Active Habit Set (v1.0)

ID	Label	Category	Daily?	Day Restriction
<code>sleep</code>	Bed by Midnight	<code>body</code>	Yes	—
<code>wake</code>	Wake by 7AM	<code>body</code>	Yes	—
<code>deep_work</code>	Deep Work Block	<code>work</code>	Yes	—
<code>applications</code>	Job Applications	<code>work</code>	Yes	—
<code>exercise</code>	Exercise	<code>body</code>	Yes	—
<code>water</code>	Water Intake	<code>body</code>	Yes	—
<code>no_phone</code>	No-Phone Morning	<code>mind</code>	Yes	—
<code>sabbath_prep</code>	Sabbath Prep	<code>spirit</code>	No	Friday only
<code>weekly_review</code>	Weekly Review	<code>mind</code>	No	Friday only

Business Logic: Day-Restricted Habits

Rule: `isHabitActiveOn(habit, date)` returns `true` iff `habit.everyDay === true`, or iff `getDow(date) === habit.dayOfWeek` where `getDow` converts JS `getDay()` to ISO Monday-origin (0=Mon, 4=Fri, 6=Sun).

Inactive cells are rendered at 8% opacity with `pointer-events: none`. They are excluded from completion percentage calculations.

Persistence Model

localStorage Schema

All state is serialised to a single key: `ht_v4`.

```
{
  "data": {
    "YYYY-MM-DD": {
```

```

    "habit_id": boolean,           // habit check-in
    "mood": 1-5,                  // daily mood rating
    "water": number,              // cups count
    "sleep_hrs": number,          // hours (0.5 step)
    "gratitude": string           // free text
  }
},
"goals": [ GoalObject ],
"mode": "dark" | "light"
}

```

Goal Object Schema

Field	Type	Description
id	string	Date.now() stringified
title	string	Goal headline
description	string	Optional success definition
duration	number	Days (7, 14, 30, 60, 90, or custom)
startDate	YYYY-MM-DD	ISO date string
linkedHabits	string[]	Array of habit IDs (may be empty)
colorIdx	0-4	Index into WEEK_PALETTE
standalone	boolean	true if no habits linked

Design Token System

Theme Tokens

The application maintains two complete token sets under `THEME.dark` and `THEME.light`.

Token	Dark Value	Light Value	Usage
bg	 #090B18	 #F4F3FF	Page background
surface	#0D0F22	#FFFFFF	Card / panel
surface2	#11142E	#EEEEFF	Input / nested bg
border	#1A1E3A	#D8D6F8	Default border
border2	#242850	#C0BEF0	Emphasis border
text	 #DDE1FF	 #1A1840	Body copy
textMuted	 #5A60A0	#6860B8	Labels, meta
textDim	#2E3462	#A098D8	Hints, timestamps
accent	 #5B54E8	 #5B54E8	Primary action
accentSoft	#5B54E822	#5B54E818	Soft accent fill

Week Colour Palette

Week bands cycle through 5 colours. Each palette entry has three variants.

Index	Name	bg	light	dim
0	Purple	 #5B54E8	#9B95FF	#1A1840
1	Teal	 #00B4CC	#40D4EC	#00303A
2	Pink	 #D4156E	#F060A0	#3A0020
3	Green	 #00C97A	#40E89A	#003822
4	Amber	 #E07B00	#F5A840	#3A2000

Typography

Role	Family	Weight	Usage
Display	Cormorant Garamond	700 italic	Month name, section headings
Body	Outfit	400 / 500	All UI text
Numeric	Outfit	700	Counts, percentages
Mono	system monospace	—	Debug / code only

Component Inventory

Component	Type	Responsibilities
HabitTracker	Root	State root, persistence, routing between panels
GridPanel	Panel	Month-week checkbox grid, summary rows
AnalysisPanel	Panel	Habit bars, week breakdown, 30d trend SVG, mood chart
TrackersPanel	Panel	Mood picker, water counter, sleep slider, Pomodoro, gratitude
GoalsPanel	Panel	Goal cards list + <code>NewGoalForm</code>
MoodPicker	Widget	5-emoji mood selector; emits 1–5 integer
PomodoroTimer	Widget	25/5 interval timer with SVG ring; session counter
GoalCard	Card	Renders goal with progress bar and linked habit chips
NewGoalForm	Form	Duration picker, habit linker, colour selector

Grid Panel Specification

Layout Dimensions

Variable	Value	Description
CELL	28px	Checkbox cell width & height
LABEL_W	168px	Habit label column width
GAP	3px	Inter-cell gap

Week Band Header

Each week occupies $\text{CELL} \times 7 + \text{GAP} \times 6$ pixels. The header band is a full-bleed coloured rectangle with:

- Week number in 9px white uppercase
- Day abbreviation row (Mo/Tu/.../Su) at 7px opacity 60%
- Date number at 9px. Today's date renders with `background: rgba(0,0,0,0.35)`, bold

Cell States

State	Class	Visual
Empty (active)	—	Transparent fill; 1.5px border at <code>pal.bg</code> + "55"
Checked	<code>done</code>	Solid <code>pal.bg</code> fill; white ; glow shadow
Today (empty)	<code>today-col</code>	Inset ring at <code>pal.light</code> ; no fill
Out of month	<code>out</code>	12% opacity; non-interactive
Day-inactive	<code>inactive</code>	8% opacity; <code>pointer-events: none</code>

Summary Row Logic

Three rows run below the grid: **Progress** (%), **Done** (count), **Not Done** (count). Each value is computed per calendar day from the active habit set for that specific day, *not* a weekly aggregate.

Daily Trackers Specification

Tracker	Key	Type	Notes
Mood & Energy	<code>mood</code>	int 1–5	5-emoji picker; verbal label displayed
Water Intake	<code>water</code>	int ≥ 0	Cup counter; 8-cup goal bar
Sleep Hours	<code>sleep_hrs</code>	float 0–12	Range slider, 0.5h step; colour-coded quality
Focus Sessions	—	in-memory	Pomodoro 25/5; session count in-memory only
Gratitude Journal	<code>gratitude</code>	string	Textarea; character count; auto-saves on change

Sleep quality thresholds:

- $\geq 7\text{h}$ → green, “Well rested ”
- 5–6.5h → amber, “Borderline — aim for 7h”
- $< 5\text{h}$ → red, “Insufficient — affects performance”

Goals Module Specification

Goal Types

Type	Behaviour
Standalone	No habit links. Duration progress bar only.
Habit-linked	Linked habit chips displayed on card. Progress bar reflects elapsed days.

Duration Options

7, 14, 30, 60, 90 days, or custom integer entry.

Progress Calculation

$$\text{progress}\% = \min\left(100, \left\lfloor \frac{\text{elapseddays}}{\text{duration}} \times 100 \right\rfloor\right)$$

where elapsed days = $\max(0, \lfloor (\text{now} - \text{startDate}) / 86400000 \rfloor)$.

Pomodoro Timer Specification

- **Work phase:** 25 minutes. On expiry, auto-transition to break phase, increment session counter.
- **Break phase:** 5 minutes. On expiry, auto-transition back to work phase. Timer stops; user restarts manually.
- **Visual:** SVG ring; `stroke-dashoffset` animates with `transition: stroke-dashoffset 1s linear`.
- **Session count:** In-memory only (not persisted). Resets on page reload by design.
- **Controls:** Start / Pause / Reset. Reset returns to current phase start time without clearing session count.

Theme Toggle Specification

- Toggle button in the top-right header, always visible.
- Mode stored in `localStorage` under the shared `ht_v4` key.
- All colour values injected via the `t` prop/variable sourced from `THEME[mode]`.
- CSS transition: `background 0.3s, color 0.3s` on root for smooth transition.
- Week palette colours are *invariant* across themes — only surface/text tokens switch.

Responsive Behaviour

Breakpoint	Width	Behaviour
Desktop	> 900px	Full two-column layout in Analysis, Trackers, Goals
Tablet	600–900px	Columns wrap via <code>flex-wrap: wrap</code>
Mobile	< 600px	Single column; grid scrolls horizontally; cells remain 28px

The monthly grid is *not* responsive — it maintains fixed cell dimensions and scrolls horizontally. This is intentional for data density.

State Management

All state is managed in the root `HabitTracker` component via `useState`. There is no context, no Redux, no external state library. The `save(data, goals, mode)` function is the single write path: it calls `setData`, `setGoals`, `setMode`, and `localStorage.setItem` atomically.

Upgrade path: Replace `save()` with an API call to persist to a database. The data shape is already well-normalised: daily records keyed by ISO date, goals as a flat array, mode as a string.

Dependency Manifest

Package	Version	Type	Purpose
<code>react</code>	18.x	runtime	UI rendering
<code>react-dom</code>	18.x	runtime	DOM mounting
Google Fonts: Outfit	CDN	asset	Body typeface
Google Fonts: Cormorant Garamond	CDN	asset	Display typeface

No chart libraries, no icon packs, no animation libraries. All visuals are native CSS + inline SVG.

Future Considerations

1. **Backend sync.** Replace `localStorage` with a REST or GraphQL API. Data model requires no structural change.
2. **Push reminders.** Web Push API for habit reminders at configurable times.
3. **Habit customisation.** Allow users to define, rename, reorder, and delete habits from within the app.
4. **Weekly email digest.** Scheduled summary of completion rates and streak data.
5. **Export.** CSV/JSON export of historical data.
6. **Streak engine.** Per-habit consecutive-day streak with visual flame indicator.

7. Offline PWA. Service worker + manifest for installable mobile experience.
